CITY COLLEGE

Approved by AICTE, New Delhi. Recognized by Govt. of Karnataka. Affiliated to Bangalore University, accredited by NAAC with "B+" Grade No. 27/2, 33rd Cross, 2nd Main Rd, 7th Block, Jayanagar, Bengaluru, Karnataka 560070 Phone: 6364914155 / 56, +91 80 26649277
Website: www.citycollege.ac.in Email: admin@citycollege.ac.in

Project Report On

"ECOSORT AI – AN AI-BASED WASTE CLASSIFICATION SYSTEM WITH A WEB DASHBOARD"

Submitted in partial fulfilment of the requirement for the project work for the degree of

MASTER OF COMPUTER APPLICATION (MCA)

2025–26

BANGALORE UNIVERSITY

Under the Guidance Of

Ms. Chandana Khatavakar

Assistant Professor

Department of Computer Application

Submitted by

NR SOWNDARYA
(P03BX25S162007)

CITY COLLEGE, JAYANAGAR, BENGALURU

DEPARTMENT OF COMPUTER APPLICATION

CERTIFICATE

This is to certify that the project synopsis entitled "ECOSORT AI – AN AI-BASED WASTE CLASSIFICATION SYSTEM WITH A WEB DASHBOARD" is submitted by Ms. NR Sowndarya (USN: P03BX25S162007) in partial fulfilment of the requirements for the award of the degree of Masters of Computer Applications (MCA) from Bangalore University for the academic year 2025–2026.

The project synopsis has been prepared under the guidance and supervision of the undersigned. The synopsis is submitted for academic evaluation in accordance with the applicable department and university guidelines.

Project Guide Ms. Chandana Khatavakar Assistant Professor	**Head of the Department** Dr. Nageshwari Department of Computer Application	**Principal** City College Jayanagar, Bengaluru

Place: Bengaluru

Date: _____

Abstract

Incorrect waste segregation lowers recycling efficiency, contaminates recyclable material and increases landfill. Many people are simply unsure which bin an item belongs in, and there is no quick way to find out at the moment of disposal.

EcoSort AI is a full-stack web application that classifies a photograph of a waste item into six categories (cardboard, glass, metal, paper, plastic and trash) and tells the user which bin to use, whether the item is recyclable and how to prepare it for disposal. A MobileNetV2 convolutional network was fine-tuned on the TrashNet dataset using transfer learning. The baseline model with a frozen backbone reached 84.2% accuracy; after partial fine-tuning, stronger data augmentation and class weights, the final model reached 92.1% accuracy on 506 unseen test images.

The model is served through a FastAPI REST back end. Users register and log in, and every classification is saved in a SQLite database against their account. A web interface provides image upload with confidence and top-3 predictions, a personal dashboard with charts, an Eco Level and estimated CO2 savings, a history page with CSV export, and a Categories page that explains how each decision is made. The application warns the user when the model's confidence is below 60%. The system runs on a normal laptop CPU with free, open-source software.

Table of Contents

Chapter 1: Introduction **5**
1.1 Background 5
1.2 Problem Statement 5
1.3 Objectives 5
1.4 Scope of the Project 5
1.5 Organisation of the Report 6
Chapter 2: Existing and Proposed System **7**
2.1 Existing System 7
2.2 Proposed System 7
Chapter 3: System Analysis **8**
3.1 Functional Requirements 8
3.2 Non-Functional Requirements 8
3.3 Feasibility Study 9
3.4 Software and Hardware Requirements 9
Chapter 4: System Design **10**
4.1 Architecture 10
4.2 Application Workflow 10
4.3 Module Description 11
4.4 Database Design 11
4.5 API Design 13
4.6 Security Design 13
Chapter 5: Technology Stack **14**
Chapter 6: Implementation **15**
6.1 Project Structure 15
6.2 Back-end Implementation 15
6.3 Model Development 16
6.4 Front-end Implementation 16
6.5 Sample Request and Response 17
Chapter 7: Testing **18**
7.1 Testing Approach 18
7.2 Test Cases 18
7.3 Model Evaluation Results 18
7.4 Real-World Observations 19
Chapter 8: Deployment and Configuration **21**
8.1 Deployment 21
8.2 Configuration 21
8.3 Running Locally 21
Chapter 9: Limitations **22**
Chapter 10: Future Enhancements **23**
Chapter 11: Conclusion **24**
References **25**
Appendix A: Screenshots **26**

Chapter 1: Introduction

1.1 Background

Waste management depends on correct segregation at the source. Households, campuses and offices usually mix recyclable and non-recyclable waste, because people do not know the rules or cannot identify the material quickly. Mixed waste contaminates recycling batches and raises the cost of sorting.

Computer vision can identify the material of an object from a photograph. Convolutional neural networks pre-trained on large image collections can be adapted to waste images with a small dataset through transfer learning. EcoSort AI combines such a model with a conventional web application, so that a user gets disposal guidance, and the system keeps a record of what was classified.

1.2 Problem Statement

There is no simple, always-available way for a person to find out which bin a waste item belongs in. Printed guides are rarely at hand, and generic search results are not tied to the actual item. There is a need for a single web application that identifies a waste item from a photo, gives the correct disposal guidance, admits when it is unsure, and keeps a history and analytics for each user.

1.3 Objectives

- To train an image classifier for six waste categories with accuracy above 90%.
- To serve the model through a REST API that returns the category, confidence, top-3 predictions, bin and disposal tip.
- To provide registration and login so that each user has a private history.
- To provide a dashboard with analytics, a history page and CSV export.
- To warn the user when the prediction confidence is low.
- To evaluate the model with standard metrics and report its limitations honestly.
- To protect user data with salted password hashing and token-based access control.

1.4 Scope of the Project

The application targets individual users such as students, households and office staff. It is delivered as one web application and one API server. The model recognises six categories from a single photograph containing one main item. Detection of several items in one photo, a mobile application, real-time camera capture and municipal bin-collection integration are outside the present scope and are listed under future enhancements.

1.5 Organisation of the Report

Chapter 2 reviews existing approaches and the proposed system. Chapter 3 presents the system analysis. Chapter 4 describes the system design. Chapter 5 covers the technology stack. Chapter 6 explains the implementation, including model development. Chapter 7 describes testing and results. Chapter 8 covers deployment and configuration. Chapters 9 and 10 list limitations and future enhancements, followed by the conclusion, references and screenshots.

Chapter 2: Existing and Proposed System

2.1 Existing System

Waste disposal guidance is currently handled in the following ways:

- **Printed guides and bin labels:** they list categories but cannot help with an unfamiliar or mixed item, and people rarely read them at the moment of disposal.
- **Manual sorting:** waste is sorted by workers after collection, which is slow, costly and unpleasant, and contaminated batches are often rejected.
- **General search or chatbots:** answers are not tied to the user's actual item, depend on how the item is described and give no record of what was disposed.

Drawbacks of the existing approaches

- No instant, item-specific guidance at the point of disposal.
- No feedback on how reliable the answer is.
- No history or analytics to encourage better habits.

2.2 Proposed System

EcoSort AI lets a registered user upload a photo of a waste item. The server resizes and normalises the image, the neural network scores all six categories, and the highest-probability category is mapped to a bin colour, a recyclable flag and a disposal tip. The three best predictions are shown with their probabilities, and a warning appears if the top probability is below 60%. Each result is saved to the user's history and feeds the dashboard.

Advantages of the proposed system

- Instant classification and bin guidance from one photo.
- Transparent confidence and top-3 predictions, with a low-confidence warning.
- Private history, charts, Eco Level and CSV export for every user.
- Salted password hashing and token-based access control.
- Runs on a laptop with free, open-source tools.

Chapter 3: System Analysis

3.1 Functional Requirements

ID	Requirement
FR1	A visitor can register with a name, email and password; invalid emails and passwords shorter than 6 characters are rejected.
FR2	A registered user can log in and receives a session token.
FR3	A logged-in user can upload an image (JPG or PNG) and receive a classification.
FR4	The result shows the category, confidence, top-3 predictions, bin, recyclable status and disposal tip.
FR5	A warning is shown when the confidence is below 60%.
FR6	Every classification is saved against the user's account.
FR7	The dashboard shows total items, recyclable share, average confidence and estimated CO2 saved.
FR8	The dashboard shows a category chart, a daily trend chart and an Eco Level.
FR9	The history page lists the latest 50 classifications.
FR10	The user can export the full history as a CSV file.
FR11	The Categories page explains how a decision is made and describes each category.
FR12	A user can only see their own data; protected routes reject requests without a valid token.

3.2 Non-Functional Requirements

- **Security:** passwords are stored as salted hashes; protected endpoints verify a token; each query is filtered by the logged-in user.
- **Usability:** a clean responsive interface, drag-and-drop upload, clear error messages and a loading state.
- **Performance:** the model is loaded once at start-up and classifies an image on a laptop CPU in about a second.
- **Reliability:** invalid or unreadable images return a clear message instead of crashing the server.
- **Maintainability:** the training script, evaluation script and server are separate files with small, documented functions.
- **Portability:** the application runs on Windows, Linux and macOS with the same code and needs no external services.

3.3 Feasibility Study

Technical feasibility: PyTorch, FastAPI and SQLite are mature, well documented and free. MobileNetV2 is small enough to train and run on a laptop CPU.

Economic feasibility: all software is open source and the dataset is public, so no licence or infrastructure cost is needed.

Operational feasibility: the application runs in a web browser without installation, and the steps (log in, upload, read the result) are simple for any user.

3.4 Software and Hardware Requirements

Item	Requirement
Operating system	Windows / Linux / macOS
Language	Python 3.9 or newer (developed on Python 3.13)
Back end	FastAPI, Uvicorn, Pydantic, python-multipart, Pillow
Machine learning	PyTorch, torchvision (MobileNetV2), scikit-learn and matplotlib for evaluation
Database	SQLite (built into Python, no installation)
Front end	HTML, CSS, JavaScript, Chart.js
Browser	Any modern browser (Chrome, Edge, Firefox, Safari)
Hardware	Any computer with at least 4 GB RAM and about 3 GB free disk space; no GPU required

Chapter 4: System Design

4.1 Architecture

EcoSort AI follows a three-tier client-server architecture. The browser runs a single-page web application. It sends JSON and image uploads over HTTP to one FastAPI server, which authenticates each request, runs the model, and reads and writes a SQLite database.

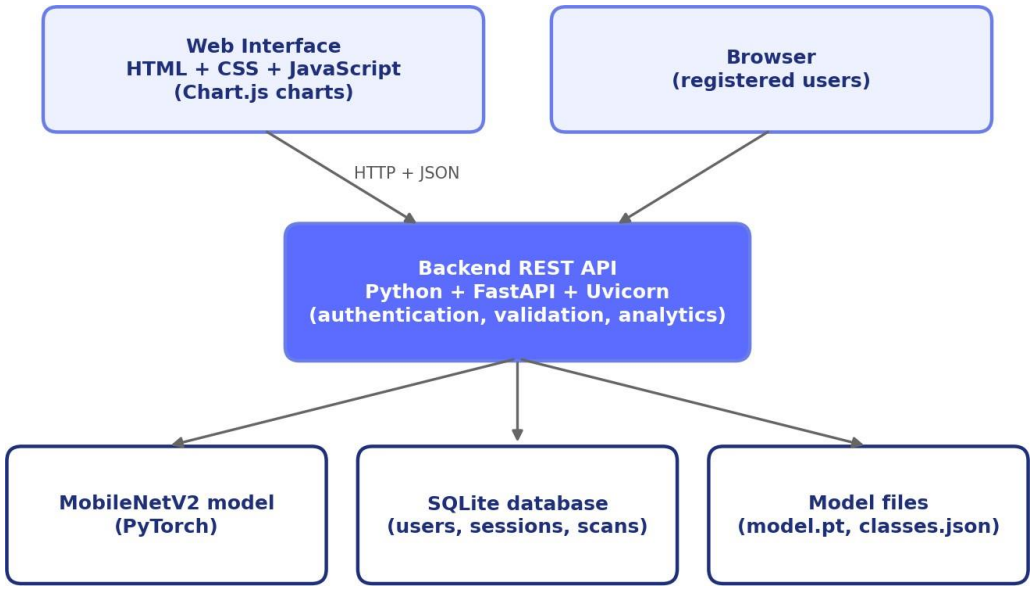


Figure 4.1: System architecture of EcoSort AI

4.2 Application Workflow

A user registers or logs in and receives a session token. On the Classify page the user uploads a photo. The server checks the token and the file type, opens the image, resizes it to 224 x 224 pixels and normalises it. MobileNetV2 outputs six scores and softmax converts them to probabilities. The top category is mapped to a bin, a recyclable flag and a tip, the result is saved in the scans table and returned as JSON. The interface shows the result, with a low-confidence warning if the top probability is below 60%, and the dashboard and history update.

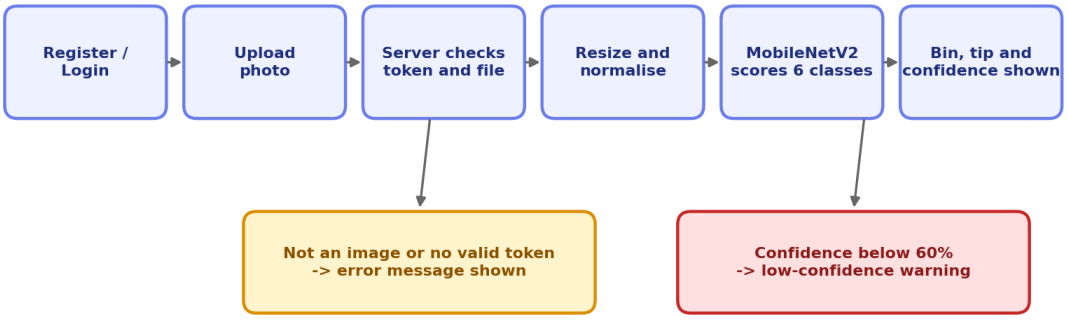


Figure 4.2: Classification workflow

4.3 Module Description

Module	Description
Authentication	Registration and login; a random token is issued and checked on every protected request.
Classification	Validates the upload, runs the model and returns the category, probabilities, bin and tip.
Analytics	Per-user totals, category counts, daily counts, average confidence and estimated CO2 saved.
History and export	Lists the latest scans and exports all of them as CSV.
Categories guide	Explains how a decision is made and describes each category and bin.
Model training and evaluation	Scripts that fine-tune MobileNetV2 and produce precision, recall, F1 and a confusion matrix.

4.4 Database Design

SQLite is a file-based relational database stored as waste.db. The application uses three tables. Sessions and scans are linked to users by user_id.

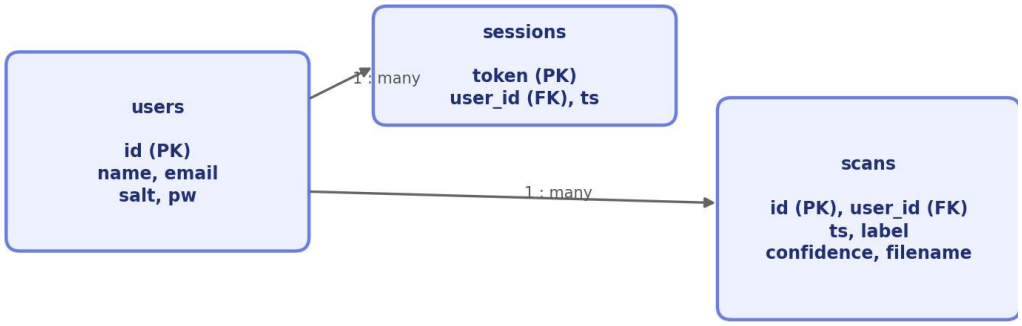


Figure 4.3: Tables and relationships

Table: users

Field	Type	Description
id	INTEGER (PK)	Auto-generated unique identifier
name	TEXT	Name of the user
email	TEXT (unique)	Email used to log in
salt	TEXT	Random per-user salt
pw	TEXT	PBKDF2-HMAC-SHA256 hash of the password (the plain password is never stored)

Table: sessions

Field	Type	Description
token	TEXT (PK)	Random session token
user_id	INTEGER (FK)	Owner of the session
ts	REAL	Time the session was created

Table: scans

Field	Type	Description
id	INTEGER (PK)	Auto-generated unique identifier
user_id	INTEGER (FK)	User who classified the item
ts	REAL	Time of classification
label	TEXT	Predicted category
confidence	REAL	Probability of the predicted category
filename	TEXT	Name of the uploaded file (the image itself is not stored)

4.5 API Design

All endpoints are under /api. Protected routes expect the header Authorization: Bearer token. FastAPI also generates interactive documentation at /docs.

Method	Endpoint	Auth	Purpose
POST	/register	No	Create an account and return a token
POST	/login	No	Verify credentials and return a token
GET	/me	Yes	Return the logged-in user
POST	/predict	Yes	Classify an uploaded image and save the result
GET	/stats	Yes	Totals, category counts, daily counts, average confidence, estimated CO2
GET	/history	Yes	Latest 50 scans
GET	/export	Yes	Download all scans as CSV

4.6 Security Design

- Passwords are hashed with PBKDF2-HMAC-SHA256 (100,000 iterations) and a random per-user salt; the plain password is never saved.
- Each login creates a random session token sent in the Authorization header.
- The user's identity comes from the verified token, not from the request body, and every data query is filtered by user_id, so one user cannot see another's data.
- Email format and password length are validated at registration, and uploaded files are checked to be readable images.
- The interface escapes user-supplied text before displaying it, which prevents script injection.

Chapter 5: Technology Stack

Layer	Technology	Reason for choice
Machine learning	PyTorch, torchvision, MobileNetV2	Standard framework; small, accurate model that runs on a CPU
Back end	Python, FastAPI, Uvicorn	Fast, typed REST API with automatic documentation
Validation	Pydantic	Checks request bodies
Image handling	Pillow	Opens and converts uploaded photos
Database	SQLite	Zero set-up, one file, suitable for a prototype
Security	hashlib (PBKDF2), secrets	Salted password hashing and random tokens from the standard library
Front end	HTML, CSS, JavaScript	No build step; easy to run and demonstrate
Charts	Chart.js	Doughnut, line and bar charts in the browser
Evaluation	scikit-learn, matplotlib	Classification report and confusion matrix

Chapter 6: Implementation

6.1 Project Structure

```
ecosort/
  backend/
    main.py          FastAPI app, authentication, inference, analytics
    train.py         model training
    evaluate.py      metrics and confusion matrix
    model.pt         trained weights
    classes.json     class names
    waste.db         SQLite database (created on first run)
  frontend/
    index.html       single-page web application
  dataset/           TrashNet images, one folder per class
  requirements.txt
```

6.2 Back-end Implementation

6.2.1 Start-up

main.py loads classes.json and model.pt once, builds a MobileNetV2 with a six-class output layer, creates the SQLite tables if they do not exist, registers the API routes and serves the frontend folder from the same address.

6.2.2 Authentication

On registration the server validates the email and password length, creates a random salt, hashes the password with PBKDF2 and stores the user. It then creates a random token, saves it in the sessions table and returns it. At login the entered password is hashed with the stored salt and compared with the stored hash using a constant-time comparison. A dependency function reads the Bearer token from the header, looks it up in the sessions table and gives each protected endpoint the logged-in user.

6.2.3 Prediction

The predict endpoint checks that the upload is an image, opens it with Pillow and converts it to RGB. The image is resized to 224 x 224 and normalised with the ImageNet mean and standard deviation. The model returns six scores, softmax turns them into probabilities, and the three highest are returned. The top category is looked up in a table of bin, recyclable flag and tip, and the scan is saved with the user's id.

6.2.4 Analytics and export

The stats endpoint groups the user's scans by category and by day, computes the average confidence and estimates CO2 saved from illustrative per-item values for recyclable categories. The history endpoint returns the latest 50 scans, and the export endpoint streams all scans as a CSV file.

6.3 Model Development

6.3.1 Dataset

TrashNet contains 3,790 labelled photographs: paper 727, glass 875, plastic 643, metal 424, cardboard 495 and trash 645. The data is imbalanced. The images were split randomly 80/20 with a fixed seed, giving about 3,790 training images and 506 test images; the same split is used in training and evaluation.

6.3.2 Baseline model (v1)

MobileNetV2 with ImageNet weights and a frozen feature extractor; a new final layer (1280 to 6) trained with the Adam optimiser (learning rate 0.001) for 8 epochs, with resizing to 224 x 224 and random horizontal flip.

6.3.3 Improved model (v2)

Three changes address the weak classes found in the baseline evaluation:

1. Partial fine-tuning: the last feature blocks (layer 14 onward) are unfrozen and trained at a lower learning rate (0.0001) than the new classifier layer (0.001), using AdamW.
2. Stronger augmentation: random resized crop, rotation up to 15 degrees and colour jitter.
3. Class weights: the loss weights each class inversely to its frequency, so the rare trash class is not ignored.

Training ran for 12 epochs with batch size 32 and the checkpoint with the best test accuracy was kept.

6.4 Front-end Implementation

The front end is one HTML file with embedded CSS and JavaScript. It has a login and register screen and, after login, a sidebar with four pages. The session token is kept in the browser's local storage and sent with every API call.

Page	**Function**
Login / Register	Account creation and sign-in with error messages
Classify	Welcome banner, drag-and-drop upload, result card with category, confidence, bin, tip, top-3 bars and low-confidence warning, recent scans and recycling tips
Dashboard	Four summary cards, category doughnut chart, daily trend chart and Eco Level progress bar
History	Table of latest scans with an Export CSV button
Categories	How a decision is made, F1 score chart per category and a card for each category

6.5 Sample Request and Response

Request to POST /api/predict (the token is sent in the Authorization header; the image is sent as a multipart form field named file). Response, with illustrative values:

```
{
  "label": "plastic",
  "confidence": 0.9231,
  "top3": [
    {"label": "plastic", "confidence": 0.9231},
    {"label": "glass", "confidence": 0.0510},
    {"label": "metal", "confidence": 0.0142}
  ],
  "bin": "Red bin",
  "recyclable": true,
  "tip": "Rinse, check the resin code, and remove caps."
}
```

Chapter 7: Testing

7.1 Testing Approach

The application was tested functionally through the web interface and the API, and the model was evaluated on held-out images with standard metrics. The screenshots in Appendix A show the application running on the local build. Section 7.2 lists the functional test cases, section 7.3 gives the model evaluation results and section 7.4 the observations on real-world photos.

7.2 Test Cases

ID	Scenario	Expected result	Status
T1	Register with valid name, email and password	Account created and user logged in	Pass
T2	Register with an invalid email or a password under 6 characters	Error message shown	Pass
T3	Register with an email that already exists	Message: already registered	Pass
T4	Log in with correct credentials	Token returned; app opens	Pass
T5	Log in with a wrong password	Message: wrong email or password	Pass
T6	Upload a valid image	Category, confidence, bin, tip and top 3 shown	Pass
T7	Upload a file that is not an image	Message asking for a JPG or PNG	Pass
T8	Upload an image with low confidence	Low-confidence warning shown	Pass
T9	Open the Dashboard after several scans	Cards and both charts show the user's data	Pass
T10	Open History and click Export CSV	Table lists scans; CSV file downloads	Pass
T11	Log in as a second user	Second user sees only their own scans	Pass
T12	Call a protected route without a token	Message: Please log in (HTTP 401)	Pass

7.3 Model Evaluation Results

Both models were evaluated on the same 506 unseen images.

Baseline (frozen backbone)

Class	Precision	Recall	F1-score
Cardboard	0.970	0.844	0.903
Glass	0.840	0.848	0.844
Metal	0.867	0.837	0.852
Paper	0.821	0.948	0.880
Plastic	0.742	0.775	0.758
Trash	0.913	0.636	0.750
Accuracy			**0.842**

Improved (fine-tuned)

Class	Precision	Recall	F1-score
Cardboard	0.987	0.961	0.974
Glass	0.843	0.971	0.903
Metal	0.953	0.942	0.947
Paper	0.964	0.914	0.938
Plastic	0.937	0.831	0.881
Trash	0.806	0.879	0.841
Accuracy			**0.921**

Accuracy rose from 84.2% to 92.1% and macro F1 from 0.831 to 0.914, and every class improved. The largest gain was plastic (F1 0.758 to 0.881). Trash recall rose from 0.636 to 0.879 because of the class weights, at the cost of lower trash precision (0.913 to 0.806). The best epoch was chosen using the same 506 images, so 92.1% is slightly optimistic; a separate validation set would give a stricter estimate.

7.4 Real-World Observations

- A photo of an aluminium can on a plain background was classified as glass with 58.9% confidence (metal was second at 14.8%); the low confidence is exactly the case the warning is designed for.
- A pile of cardboard boxes was classified as paper at 87.8% (cardboard 12.1%). Both go in the same bin, so the disposal advice was still correct.

These cases show domain shift: TrashNet photos mostly show one item on a plain background, so photos that look different from the training data are harder, and reflective cans can resemble glass.

Chapter 8: Deployment and Configuration

8.1 Deployment

The application is built to run on a single computer. The back end serves the front end, so only one process is needed. The source code can be kept in a GitHub repository ([repository link]). For a public demonstration it can be hosted on a free Python service such as Render, and for larger use the database can be moved to PostgreSQL or MySQL.

8.2 Configuration

No environment variables or external API keys are needed. The server requires two files created by training, backend/model.pt and backend/classes.json, and creates the SQLite file backend/waste.db automatically on first run.

8.3 Running Locally

```
# Create and activate a virtual environment (Windows)
python -m venv venv
venv\Scripts\activate

# Install libraries
pip install -r requirements.txt

# Train the model (once; place TrashNet in dataset/ first)
python backend/train.py

# Evaluate (optional)
python backend/evaluate.py

# Start the server, then open http://127.0.0.1:8000
uvicorn backend.main:app --reload
```

Chapter 9: Limitations

- The dataset is small (3,790 images) and covers only six categories.
- Accuracy is lower on cluttered or unusual photos than on TrashNet-style images (domain shift).
- The model expects one main item per photo and cannot sort a pile.
- Bin colours and recyclability rules follow a project convention and differ between cities.
- The CO2 savings shown on the dashboard are illustrative estimates, not measured values.
- Session tokens do not expire and are kept in the browser's local storage; the app uses plain HTTP and has no rate limiting on login.
- SQLite allows only one writer at a time, so it is suitable for a prototype, not a large deployment.
- Uploaded images are not stored, so earlier photos cannot be reviewed.
- The project has no automated test suite; testing was done manually.

Chapter 10: Future Enhancements

- Add real-world photos and larger datasets such as TACO to reduce domain shift.
- Add object detection so that several items in one photo can be located and classified.
- Add webcam or mobile camera capture and a mobile application.
- Add more categories such as e-waste, organic and hazardous waste.
- Add explainability (Grad-CAM heat maps) to show which part of the image drove the decision.
- Add token expiry, HTTPS, login rate limiting and an admin dashboard.
- Move to a production database and deploy with Docker.
- Add automated unit and integration tests.

Chapter 11: Conclusion

EcoSort AI shows a complete machine learning application, from dataset to web interface. Fine-tuning MobileNetV2 with stronger augmentation and class weights improved accuracy from 84.2% to 92.1%, and the evaluation identified plastic and trash as the hardest categories. The system adds registration and login, per-user history, a dashboard with charts and an Eco Level, CSV export and an explanation of how each decision is made, and it reports its own uncertainty with a low-confidence warning.

The project covered the full development cycle: problem analysis, dataset preparation, model training and evaluation, REST API design, database design, secure authentication and a responsive front end. Testing on real-world photos exposed the main limitation, domain shift, which together with the other limitations gives a clear roadmap for future work.

References

- M. Yang and G. Thung, TrashNet dataset, Stanford CS229 project - https://github.com/garythung/trashnet
- M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, MobileNetV2: Inverted Residuals and Linear Bottlenecks, CVPR 2018.
- PyTorch documentation - https://pytorch.org/docs
- FastAPI documentation - https://fastapi.tiangolo.com
- SQLite documentation - https://www.sqlite.org/docs.html
- Chart.js documentation - https://www.chartjs.org/docs
- scikit-learn documentation - https://scikit-learn.org

Appendix A: Screenshots

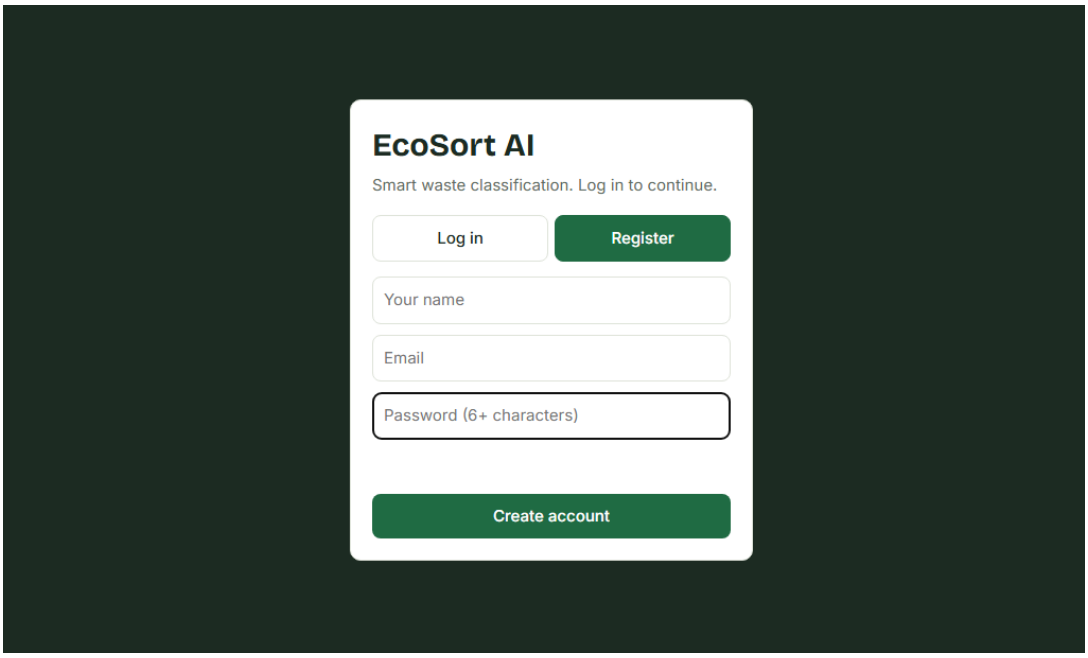


Figure A.1: Register page

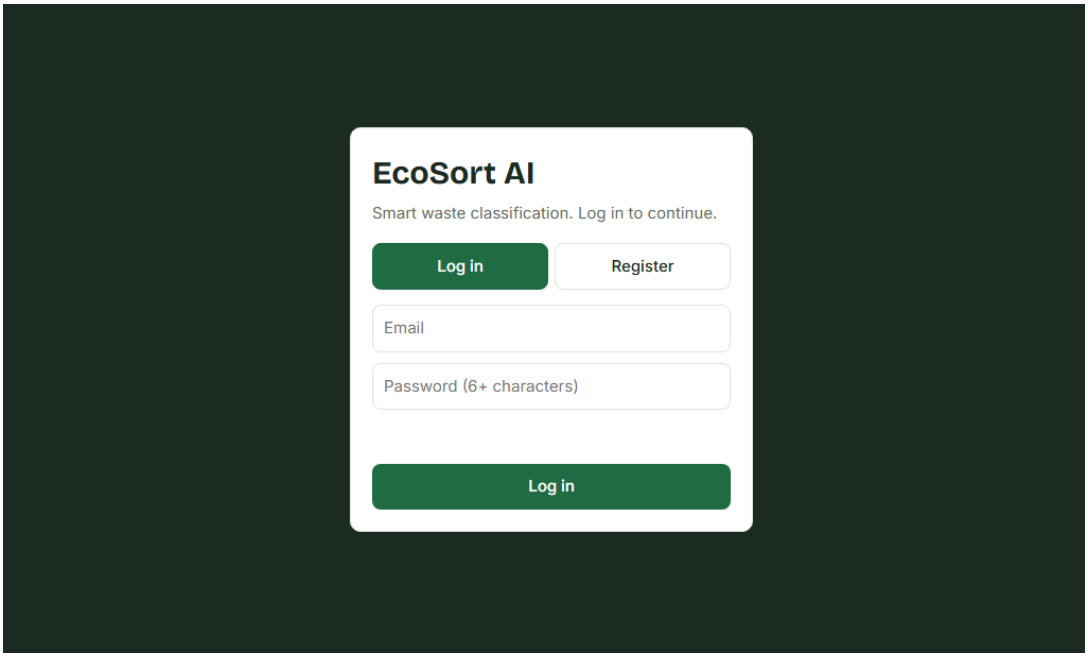


Figure A.2: Login page

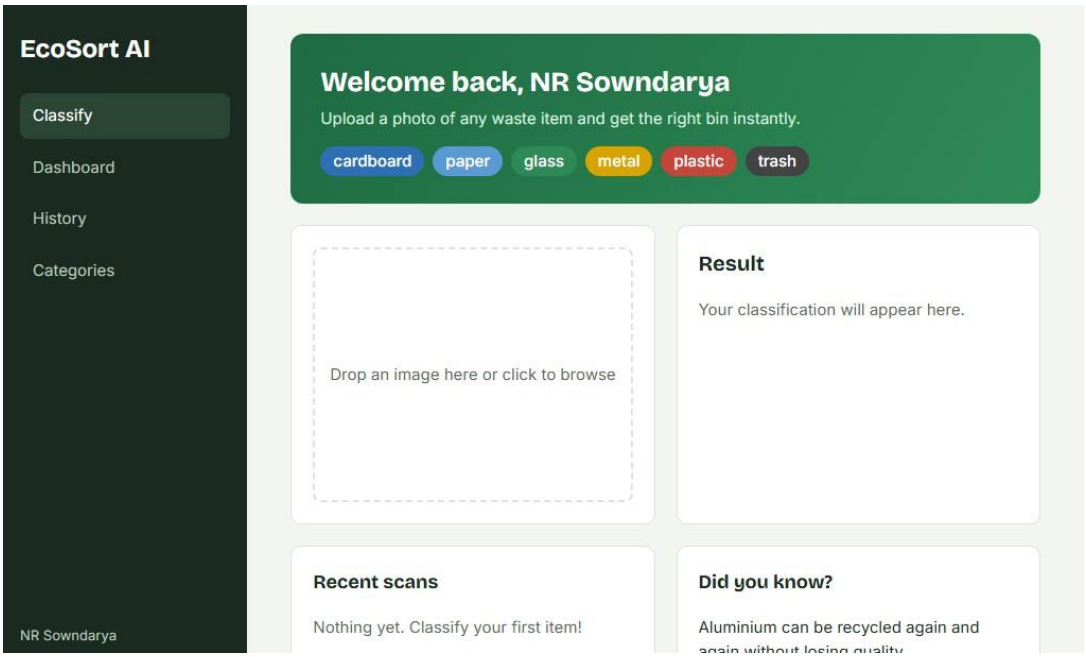


Figure A.3: Classify page with upload area, recent scans and tips

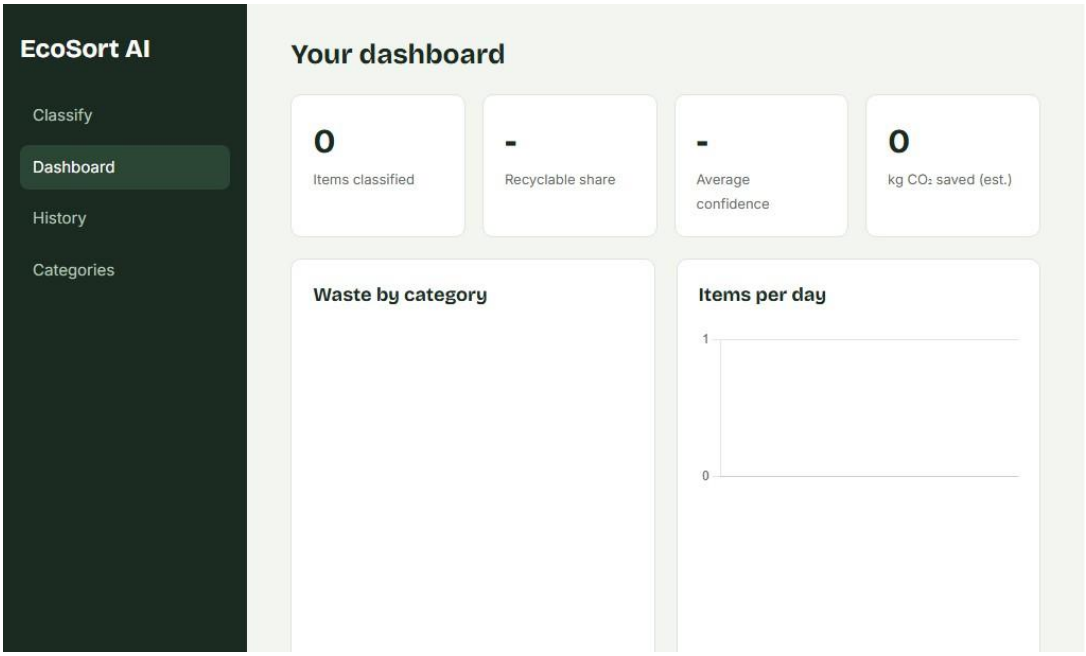


Figure A.4: Dashboard with summary cards and charts

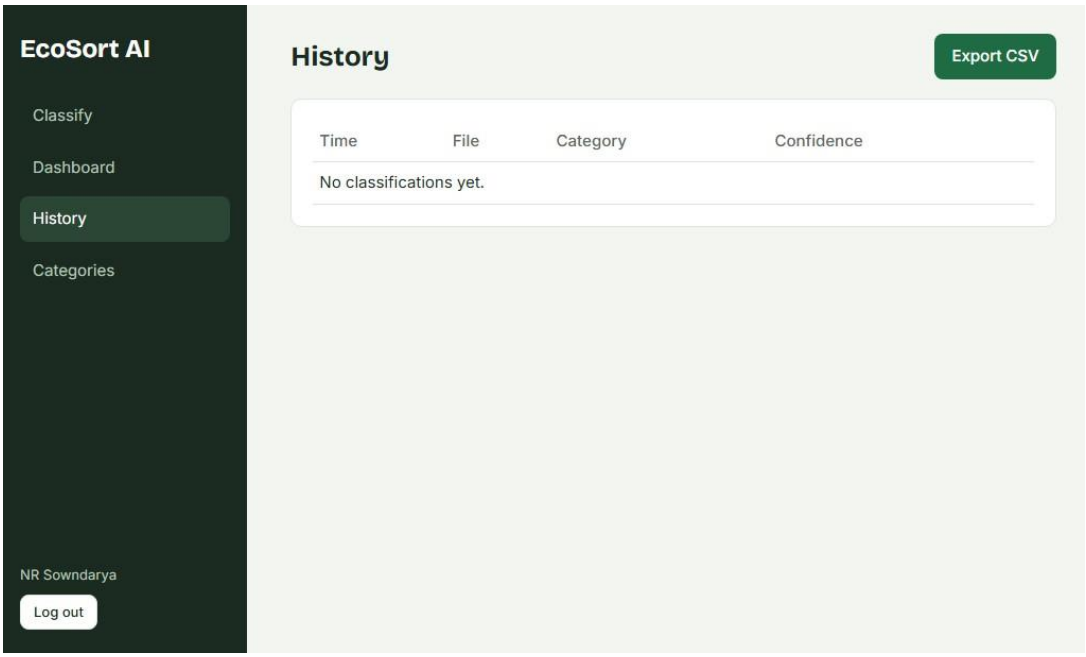


Figure A.5: History page with Export CSV

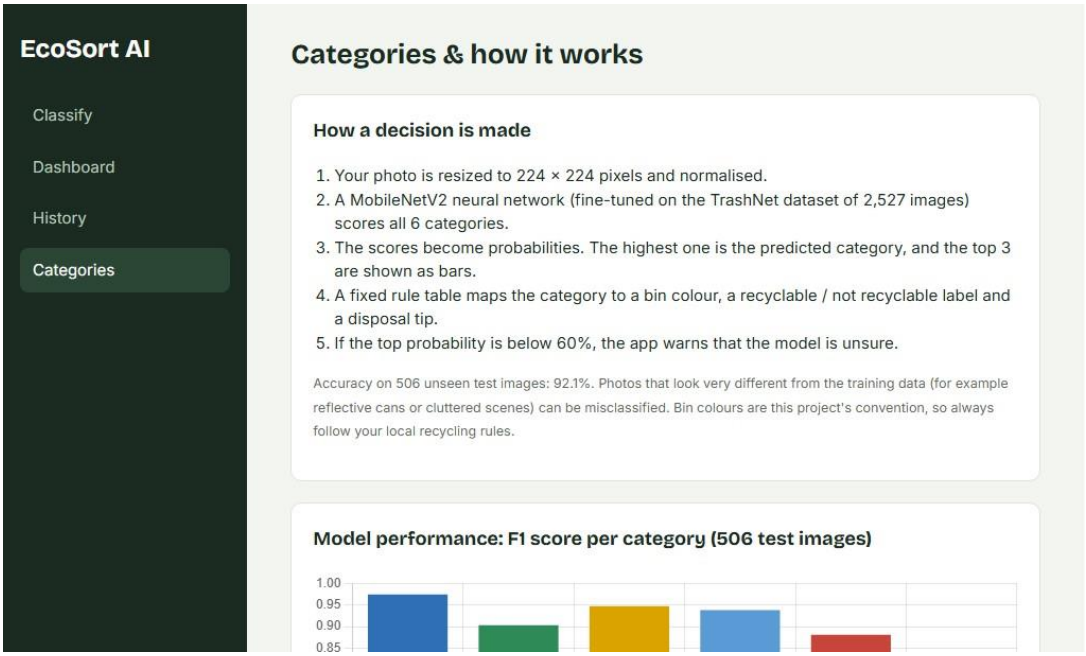


Figure A.6: Categories page - how a decision is made

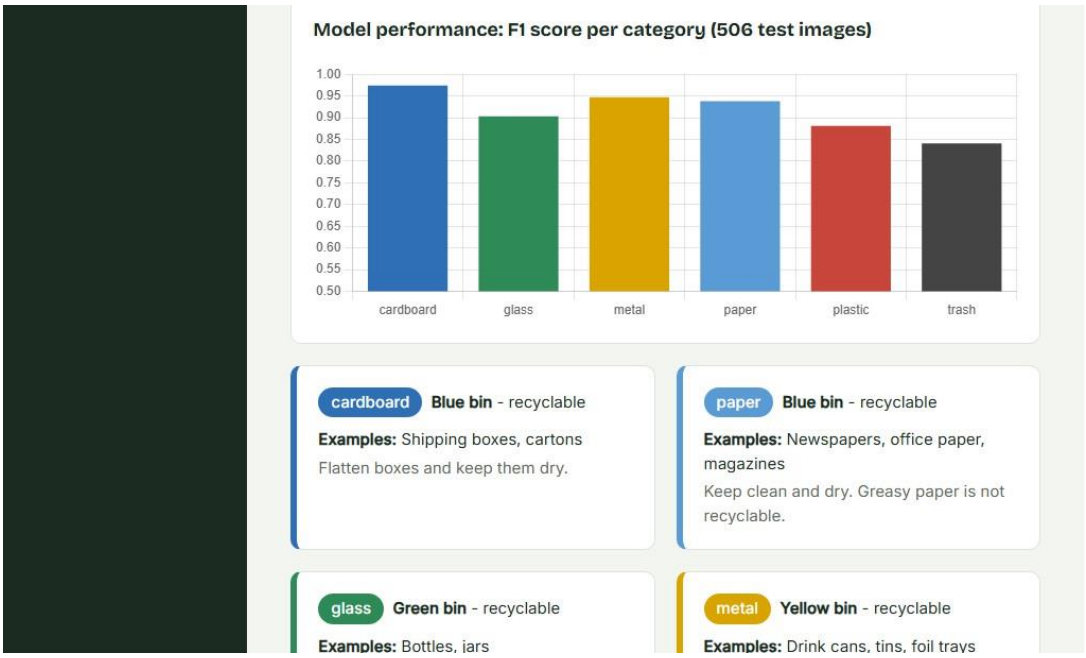


Figure A.7: Categories page - F1 score per category

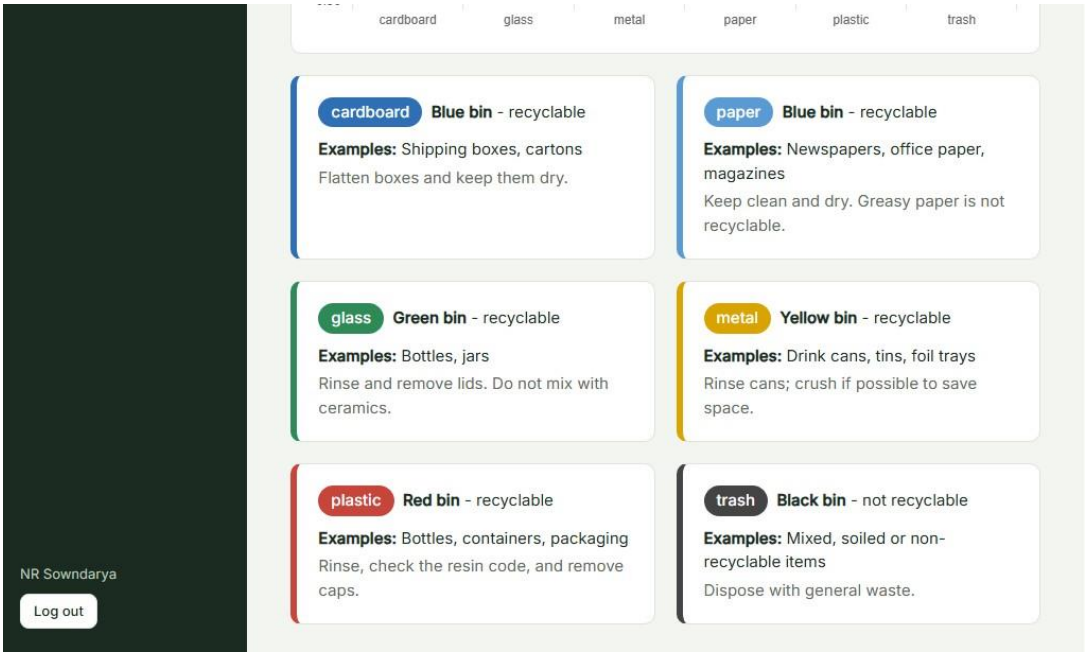


Figure A.8: Categories page - the six categories and bins

Submitted by: NR SOWNDARYA